

JAVA SDE-2 INTERVIEW PREPARATION SERIES

FRAMEWORKS, DATA, AND MESSAGING - BOOK 07 OF 12 - STUDY STEP FW-07

MongoDB for Java Backend Interviews

Document Modeling, Indexes, Consistency, Aggregation, and Spring Integration

BY

Vinay Reddy Kalluri

A modeling-first guide to document databases, with Java driver and Spring Data integration placed after storage semantics.

MONGODB | DOCUMENTS | INDEXES | AGGREGATION | CONSISTENCY

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to FW 06 – Spring Data. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Teach when document storage fits and how schema, indexing, consistency, and access patterns drive the design.

Readiness map

Decision	Evidence
START HERE	Data is accessed as aggregates or documents; Embedding and referencing create different consistency costs; An aggregation or index must support a concrete access pattern.
PREPARE FIRST	FW-01 MySQL for relational comparison; FW-06 Spring Data recommended.
MOVE ON WHEN	Model document aggregates from access patterns and atomicity boundaries; Design indexes, aggregations, consistency, and sharding from evidence; Integrate Java and Spring Data with explicit failure and retry contracts.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Frameworks Segment Position

FW 06 - Spring Data	YOU ARE HERE FW 07 - MongoDB	FW 08 - Redis
---------------------	---	---------------

A note from Vinay

Document modeling starts with the read and update boundaries, not with translating tables into JSON. Decide what must change atomically and what can grow before choosing embed or reference.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - MongoDB for Java Backend Interviews	4
Chapter 2 - BSON, CRUD, Validation, Nulls, Arrays, and Update Semantics	7
Chapter 3 - Document Modeling: Embedding, References, Growth, and Atomicity	10
Chapter 4 - Indexes, Multikey Behavior, Query Plans, and Pagination	14
Chapter 5 - Aggregation Pipelines, Memory, and Result Grain	17
Chapter 6 - Replication, Read/Write Concerns, Transactions, and Retries	20
Chapter 7 - Sharding, Change Streams, Migrations, Operations, and Security	23
Chapter 8 - Java Driver and Spring Data Boundaries, Testing, and Diagnostics	27
Chapter 9 - MongoDB Live Interviews, Rapid Q&A, Practice, and Sources	30
Chapter 10 - Java 21 MongoDB Interview Reasoning Companion	34
Part II - Practice and Handoff	37

Part I - Core Learning**SDE-2 CORE CHAPTER**

Chapter 1 - MongoDB for Java Backend Interviews

WHY IT WORKS | Learning Path and Document-First Principles

MongoDB stores BSON documents in collections. That sentence is simple; the engineering work is deciding which facts belong in one atomic document, which access patterns need indexes, and which consistency/durability guarantees the application actually requests.

From Vinay: Do not choose MongoDB because JSON feels natural in JavaScript or because "the schema changes often." Begin with the operations the service must make correct. A document is valuable when it is the unit you read, update, and protect together.

What this book builds

TEXT

```
requirements + access patterns
-> document boundary and validation
-> CRUD and atomic update shape
-> index and query-plan evidence
-> aggregation and pagination
-> replication + read/write concern
-> transactions and retryable failure
-> sharding and operations
-> Java driver / Spring Data boundary
```

The MySQL volume taught relational invariants. This volume keeps that discipline while changing the model: some relationships become embedded arrays or subdocuments; others remain references because they grow independently or have different lifecycle/ownership.

One running domain

JAVASCRIPT

```
{
  _id: ObjectId("..."),
  customerId: "c-42",
  status: "CREATED",
  version: NumberLong(0),
  createdAt: ISODate("2026-08-02T10:00:00Z"),
  lines: [
    { sku: "B00K-1", quantity: 2, unitPriceCents: NumberLong(2500) }
  ],
  totals: { subtotalCents: NumberLong(5000), currency: "USD" },
  requestKey: "req-8fd"
}
```

The order and its lines are often one aggregate: they are read together, have bounded growth, and must change atomically. Customer details are referenced because the customer has an independent lifecycle. `unitPriceCents` is a purchase-time fact, not an accidental copy of the current catalog price.

Single-document atomicity first

MongoDB writes are atomic at one document. Encode state preconditions in an update:

JAVASCRIPT

```
db.orders.updateOne(
  { _id: orderId, status: "CREATED", version: 3 },
  {
    $set: { status: "PAID", paidAt: new Date() },
    $inc: { version: 1 }
  }
)
```

`matchedCount: 0` is not success. It means missing document, wrong state, or stale version. Read only when the API must distinguish those cases.

The SDE-2 answer frame

- 1 **Access pattern:** What does one read/write request?
- 2 **Boundary:** Which fields must change atomically and how can the document grow?
- 3 **Index:** Which filter and sort must be supported?
- 4 **Consistency:** What read concern, write concern, and read preference are required?
- 5 **Failure:** Can a retry duplicate or reorder a side effect?
- 6 **Scale:** What is the shard key and hot-key distribution?
- 7 **Operations:** How do we migrate, observe, back up, and restore?

First failure matrix

Event	What is known	Safe response
duplicate key	unique invariant rejected write	return conflict/idempotent prior result
network error during write	outcome may be unknown	retry only retryable/idempotent operation or reconcile by key
primary steps down	in-flight operation can fail	driver discovers new primary; classify and bound retry
secondary read lags	read can omit latest write	use primary/causal session/stronger contract where required
document hits size/growth limit	update fails or model degrades	bound arrays and split independent history

Quick check and practice

- 1 Why is "schema flexible" not "schema absent"?
 - 2 Which guarantee comes for free inside one document?
 - 3 Why should an update contain the expected state/version?
- **Foundation:** List three reads and three writes for the order domain.
 - **Interview Core:** Mark which facts belong inside the order and which do not.
 - **SDE-2 Follow-up:** Reconcile a timed-out order creation using a unique request key.

SDE-2 CORE CHAPTER

Chapter 2 - BSON, CRUD, Validation, Nulls, Arrays, and Update Semantics

BSON is typed binary data

BSON distinguishes strings, booleans, dates, binary values, documents, arrays, object identifiers, and several numeric types. A Java `long` should not silently become a double when exact range matters. Choose the driver codec/mapping deliberately and test legacy documents containing older types.

Field order usually should not carry domain meaning. Document equality and some command behavior can be order-sensitive, so avoid treating raw BSON as an unordered Java map in low-level tests.

Missing and null are different states

JAVASCRIPT

```
{ nickname: null }  
{ /* nickname is missing */ }
```

`{ nickname: null }` can match both explicit null and absent field in common query semantics. Use `$exists` when the distinction matters:

JAVASCRIPT

```
{ nickname: { $exists: true, $eq: null } }
```

Decide what each state means during schema evolution. A default applied only by new Java code does not repair older documents.

WHY IT WORKS | Create and unique invariants

JAVASCRIPT

```
db.orders.createIndex(  
  { customerId: 1, requestKey: 1 },  
  { unique: true, name: "uq_order_request" }  
)
```

An existence query before insert is race-prone. The unique index arbitrates concurrent claims. For optional unique fields, learn how null/missing values interact with sparse or partial indexes and express the intended subset explicitly.

Projection and sort

JAVASCRIPT

```
db.orders.find(
  { customerId: "c-42", status: "PAID" },
  { status: 1, totals: 1, createdAt: 1 }
).sort({ createdAt: -1, _id: -1 }).limit(50)
```

The `_id` tie-breaker makes order stable. Projection reduces transfer and accidental exposure, but inclusion/exclusion rules and `_id` defaults need explicit testing.

Operators versus replacement

This updates selected fields:

JAVASCRIPT

```
db.orders.updateOne(
  { _id: id },
  { $set: { status: "CANCELLED" }, $inc: { version: 1 } }
)
```

A replacement operation replaces the document apart from `_id`; omitted fields disappear. Java object replacement can therefore delete fields written by newer services. Prefer targeted operators for commands unless full replacement is the explicit contract.

Array operations

JAVASCRIPT

```
db.orders.updateOne(
  { _id: id, "lines.sku": { $ne: "BOOK-1" } },
  { $push: { lines: { sku: "BOOK-1", quantity: 1 } } }
)
```

This shape is not a universal uniqueness guarantee under every concurrent/model condition. A document-level filter plus update is atomic, but array uniqueness should be modeled and tested carefully; an object keyed by SKU or separate collection with a unique compound index can be clearer.

Use positional operators deliberately:

- `$` updates the first matched element;
- `$[]` targets all elements;
- `[$[identifier]]` uses `arrayFilters`.

Accidentally applying `$[]` is a high-impact bug. Always test empty arrays, duplicate matching elements, and missing paths.

Upsert race reasoning

upsert: true combines match/update-or-insert, but concurrent upserts need a unique index matching the logical key. Otherwise several distinct documents can be inserted. The inserted document is derived from equality filter fields plus update operators; do not assume every query operator becomes stored state.

Schema validation

Collection validators can require types and fields at the database boundary. Roll them out compatibly: measure existing violations, add warning/moderate behavior if appropriate, backfill, then tighten. Driver POJO validation and Bean Validation improve errors but do not protect writes from other clients.

WATCH OUT | Edge cases

Case	Risk	Safer reasoning
missing versus explicit null	ambiguous query/results	<code>\$exists</code> plus migration contract
replacement from stale DTO	erases unknown fields	targeted updates and version filter
unbounded <code>\$push</code>	16 MiB limit and growing write cost	cap/window or separate event collection
floating money	precision loss	integer minor units/decimal128 with policy
concurrent upsert	duplicates	unique index on logical key
positional update	wrong/all elements changed	precise match and array filters

Practice and solution direction

- **Foundation:** Query only documents with an explicit null nickname. Use `$exists: true` and `$eq: null`.
- **Interview Core:** Make order creation idempotent. Use a scoped unique request key and interpret duplicate key as prior claim.
- **SDE-2 Follow-up:** Evolve `status` from free text to a validated set. Inventory, dual-compatible deployment, backfill, validation tightening, then remove compatibility.

SDE-2 CORE CHAPTER

Chapter 3 - Document Modeling: Embedding, References, Growth, and Atomicity

MongoDB modeling begins with operations, not entity diagrams. The same real-world relationship may be embedded, referenced, duplicated as a snapshot, or represented by a separate event collection depending on lifecycle and access.

Embed when the boundary agrees

Embedding fits when children:

- are normally read with the parent;
- have bounded count/size;
- share lifecycle and ownership;
- must change atomically with the parent;
- are not independently queried or shared heavily.

Order lines fit many of these properties. An unbounded chat history does not.

Reference when facts evolve independently

Reference when data:

- has independent lifecycle/authorization;
- is shared by many aggregates;
- grows without a safe bound;
- is queried independently;
- changes at a different frequency.

Avoid recreating relational normalization automatically. A reference means the application may issue another query/aggregation, handle absence, and accept consistency between documents.

Deliberate duplication

An order can retain `shippingAddressSnapshot` and `unitPriceCents`. They represent facts at purchase time. For a duplicated current customer name, define:

- 1 authoritative source;
- 2 acceptable staleness;
- 3 propagation mechanism;
- 4 reconciliation and repair;
- 5 behavior during partial failure.

Growth patterns

Unbounded arrays

Embedding every event, follower, or reading eventually hits document-size limits and makes each update/read progressively heavier. Use:

- bounded recent window plus archive collection;
- bucket pattern by time/count;
- child collection with parent ID and index;
- summary counters with recoverable source events.

Hot documents

A single global counter serializes writes and concentrates replication. Stripe counters across buckets and sum them when approximate/mergeable semantics permit, or choose another design. Do not shard away an invariant without defining how it is restored.

One-to-many choices

Shape	Write/read effect	Best fit
child embedded in parent	one atomic write/read, parent grows	bounded owned aggregate
parent ID on child	independent writes, extra query	large/unbounded children
child IDs in parent	parent array grows, extra lookup	bounded references where list ownership matters
duplicated summary	fast read, propagation needed	explicit staleness budget

Atomic command design

Use update operators and predicates instead of load-modify-replace:

JAVASCRIPT

```
db.inventory.updateOne(  
  { sku: "BOOK-1", available: { $gte: 2 } },  
  { $inc: { available: -2, version: 1 } }  
)
```

This prevents overselling for one inventory document. A multi-SKU reservation spans documents and requires either a transaction, compensating reservation state, or a redesigned aggregate. State the cost and failure behavior.

Multi-document transactions

Transactions exist for replica sets and sharded clusters, but they add coordination, runtime/resource limits, and failure/retry complexity. They should preserve a real cross-document invariant-not rescue a model that needlessly split one aggregate.

The driver transaction callback may retry, so the body must not send emails or call payment APIs without idempotency. Use an outbox/state document and perform external work after commit.

Polymorphic document schemas

Different document shapes in one collection can simplify queries when they share access and indexes. They can also create sparse fields, branching mapping code, and broad validation. Include a [schemaVersion](#) or stable discriminator when migration/decoding needs it; do not make every read support indefinite historical shapes.

Modeling failure matrix

Symptom	Modeling cause	Repair
document near size limit	unbounded embedded history	bucket/archive/separate children
one counter dominates latency	hot document	shard/stripe or change invariant
many round trips	over-referencing aggregate	embed bounded owned data
massive rewrites	frequently changed value duplicated everywhere	reference or async projection with repair
transaction everywhere	aggregate split by table-thinking	reconsider boundary

Practice

- **Foundation:** Model blog post and last 20 comments; explain where older comments go.
- **Interview Core:** Model product catalog price versus order price snapshot.
- **SDE-2 Follow-up:** Design a 100-million-member community without one unbounded member array.

Solution direction

Keep bounded recent comments or reference comment documents; persist agreed order price in the order; represent memberships as separate documents keyed/indexed by community and member, then page them and maintain repairable counts.

SDE-2 CORE CHAPTER

Chapter 4 - Indexes, Multikey Behavior, Query Plans, and Pagination

Design from the query shape

JAVASCRIPT

```
db.orders.find({
  customerId: "c-42",
  status: "PAID",
  createdAt: { $lt: boundaryTime }
}).sort({ createdAt: -1, _id: -1 }).limit(50)
```

A candidate:

JAVASCRIPT

```
db.orders.createIndex(
  { customerId: 1, status: 1, createdAt: -1, _id: -1 },
  { name: "ix_order_customer_status_cursor" }
)
```

Equality fields, sort, and range are a useful starting heuristic. Actual key order depends on selectivity, sort direction, query variants, and write/index cost. Validate with representative data and `explain("executionStats")`.

Compound-prefix reasoning

The order of fields defines usable prefixes. An index beginning with `customerId` cannot generally seek a query only by `status` as though status were first. Later fields can support filtering/coverage even when they no longer narrow the primary scan; read the winning plan.

Multikey indexes

Indexing an array creates multikey index entries. One document can contribute many keys, increasing size/work. Compound multikey restrictions and correlation matter.

For an array of documents:

JAVASCRIPT

```
{ lines: [ { sku: "A", quantity: 1 }, { sku: "B", quantity: 10 } ] }
```

These independent predicates may match different elements:

JAVASCRIPT

```
{ "lines.sku": "A", "lines.quantity": { $gte: 10 } }
```

Use `$elemMatch` when both must hold for the same element:

JAVASCRIPT

```
{ lines: { $elemMatch: { sku: "A", quantity: { $gte: 10 } } } }
```

Specialized index decisions

- **Unique:** database invariant; understand missing/null and sharded constraints.
- **Partial:** indexes documents matching an expression; query must be compatible for use.
- **Sparse:** omits documents without the field; not identical to partial semantics.
- **TTL:** background expiration for date fields; deletion is not immediate or an exact scheduler.
- **Text/wildcard/geospatial:** solve specific query shapes with limits and operational cost.

TTL should not enforce a security deadline by itself. Check expiry in application authorization because the cleanup monitor can run later.

Reading an explain plan

Ask:

- Did the winning plan use **IXSCAN** or **COLLSCAN**?
- Which bounds were applied to each key?
- How do keys/documents examined compare with rows returned?
- Was an in-memory/blocking sort required?
- Were documents fetched after index scan, or was the query covered?
- Did a shard router target one shard or scatter?

An index scan is not automatically efficient; scanning 10 million keys to return 10 rows is evidence of mismatch.

Internal request path

TEXT

```
Java filter + options
-> wire command
-> parser/planner + plan cache
-> index scan / collection scan
-> WiredTiger cache pages and storage
-> fetch/filter/sort/limit
-> BSON batches over cursor
-> Java decoding/mapping
```

Latency can arise in pool/server selection, network, plan/execution, storage cache, cursor batches, or decoding. Do not add an index until you locate the time and work.

Cursor pagination

For descending (`createdAt`, `_id`):

JAVASCRIPT

```
{
  customerId: "c-42",
  status: "PAID",
  $or: [
    { createdAt: { $lt: boundaryTime } },
    { createdAt: boundaryTime, _id: { $lt: boundaryId } }
  ]
}
```

Use an opaque cursor containing every ordering component. Deep `skip()` still performs/discards earlier work and shifts under concurrent writes.

WATCH OUT | Edge cases and practice

Trap	Result
index every field	write amplification/cache pressure
TTL as exact timer	late cleanup and stale authorization
array predicates without <code>\$elemMatch</code>	matches different elements
unstable sort	duplicate/missing cursor rows
plan tested on tiny uniform data	hides skew/cardinality

- **Foundation:** Explain why an index prefix matters.
- **Interview Core:** Correct the array query using `$elemMatch`.
- **SDE-2 Follow-up:** Diagnose a plan with 2M keys and 20 returned documents.

SDE-2 CORE CHAPTER

Chapter 5 - Aggregation Pipelines, Memory, and Result Grain

An aggregation pipeline transforms a stream of documents stage by stage. As in SQL, define the result grain before writing stages.

A readable pipeline

JAVASCRIPT

```
db.orders.aggregate([
  { $match: {
    status: "PAID",
    createdAt: { $gte: ISODate("2026-08-01T00:00:00Z") }
  }},
  { $group: {
    _id: "$customerId",
    paidCents: { $sum: "$totals.subtotalCents" },
    orderCount: { $sum: 1 }
  }},
  { $match: { paidCents: { $gte: NumberLong(100000) } } },
  { $sort: { paidCents: -1, _id: 1 } },
  { $limit: 100 }
])
```

The first `$match` can use an index and reduces the stream before grouping. The second filters groups. The optimizer can reorder/coalesce some stages, but write for correct semantics and verify the explain plan.

Core stages

- `$match`: filter documents; early selective filters reduce work.
- `$project/$set`: shape or compute fields; avoid discarding a field needed later.
- `$unwind`: one output per array element; can multiply rows dramatically.
- `$group`: collapse by key and accumulate.
- `$sort/$limit`: order and bound; index support depends on preceding stages.
- `$lookup`: join-like access to another collection; watch fan-out and shard behavior.
- `$facet`: multiple subpipelines over the same input; memory/work can multiply.
- `$setWindowFields`: window computations while retaining row/document grain.

\$unwind multiplication

An order with 5 lines becomes 5 pipeline documents. If a later `$lookup` returns 3 promotions per line, the stream can become 15 records for one order. Aggregating money at that point may triple count it. Track grain after every expanding stage.

\$lookup is not free embedding

`$lookup` can be appropriate for reporting or bounded joins with indexed foreign fields. It does not make document boundaries irrelevant. Ask how many local documents, how many matches per local row, whether the foreign predicate is indexed, and what transfers between shards/nodes.

Memory and spilling

Blocking stages such as sort/group may require memory and can spill to disk depending on limits/options/version. Disk allowance prevents some failures but does not make an unbounded pipeline cheap. Bound the time range, filter early, index entry stages, precompute repairable summaries, or run analytical work elsewhere.

Result materialization

`$merge` and `$out` can materialize results. Define idempotency, key choice, replacement/merge behavior, concurrent runs, and partial failure. A derived collection needs lineage, freshness, reconciliation, and rebuild procedures.

Window example: latest order per customer

JAVASCRIPT

```
[
  { $setWindowFields: {
    partitionBy: "$customerId",
    sortBy: { createdAt: -1, _id: -1 },
    output: { rank: { $documentNumber: {} } }
  }},
  { $match: { rank: 1 } }
]
```

Compare with sort-then-group patterns under your version and indexes. The best expression is the one whose semantics and plan are proven.

Failure/edge matrix

Symptom	Likely cause	Repair
totals multiplied	unwind/lookup changed grain	regroup at intended identity
memory error/spill	unbounded sort/group/facet	filter, index, partition, precompute
lookup slow	fan-out/missing foreign index	model/index/bound relation
report stale	materialized projection lag	freshness contract and reconciliation
timeout but server continues	cancellation/outcome unclear	<code>maxTimeMS</code> , operation tracking, idempotent writes

Practice with solutions

- **Foundation:** Sum paid totals by customer. `$match`, `$group`, deterministic `$sort`.
- **Interview Core:** Find top SKU without double-counting order totals. Unwind lines and sum line-level extended price, not order total.
- **SDE-2 Follow-up:** Design a daily revenue projection. Use a stable day/customer key, idempotent `$merge` policy, watermark, late-data correction, and source reconciliation.

SDE-2 CORE CHAPTER

Chapter 6 - Replication, Read/Write Concerns, Transactions, and Retries

Replica sets provide redundancy and elections. A primary accepts ordinary writes; secondaries replicate the operation log and apply changes. Failover changes which member is primary and can interrupt in-flight operations.

Internal write flow

TEXT

```
Java driver selects primary
-> command reaches primary
-> document/index changes in WiredTiger transaction
-> journal/checkpoint machinery supports recovery
-> oplog record replicated to secondaries
-> write concern decides acknowledgement point
-> response or network failure reaches client
```

The exact storage timing/configuration varies. Do not equate "acknowledged" with "every secondary has applied it" or "data file page flushed everywhere." State the selected write concern and journaling policy.

Write concern

Write concern controls required acknowledgement. Majority acknowledgement generally protects against rollback through ordinary elections better than a single-member acknowledgement, at latency/availability cost. Timeouts limit waiting but can yield an **unknown outcome**: the write may later satisfy the requested acknowledgement even though the client received a timeout.

Use a unique business/request key so retry or reconciliation can discover the result.

Read preference and read concern are separate

- **Read preference** chooses eligible members: primary, primary-preferred, secondary, and variants.
- **Read concern** controls consistency/isolation guarantees of returned data.

Reading from a secondary can reduce some source read load or place reads geographically, but it can be stale. **majority** read concern means majority-committed data, not necessarily the newest operation currently known anywhere. **linearizable** is a stronger single-document real-time read contract with restrictions/cost and should use a time bound. **snapshot** serves point-in-time semantics in supported contexts.

Do not memorize settings without a product requirement. Order confirmation may need primary/causal read-after-write; analytics may tolerate secondary lag.

Causal sessions

A causally consistent session can preserve relationships such as read-your-writes and monotonic reads when used with compatible concerns. The session context/operation time must flow with the logical request. A random next request on another session does not inherit causality.

Retryable reads and writes

Modern drivers can retry selected operations after transient network/election failures. Retryability is not permission to repeat arbitrary application workflows. A driver can retry a supported single write with an operation identity; it cannot make an email, HTTP call, and database change atomic.

Classify errors using driver labels such as transient transaction or unknown commit results per current driver guidance. Bound application retries and record attempts/latency.

Transactions

JAVA

```
session.withTransaction(() -> {  
    orders.updateOne(session, orderFilter, orderUpdate);  
    outbox.insertOne(session, eventDocument);  
    return null;  
});
```

The callback can run more than once. Keep it deterministic and database-only, or make external behavior idempotent outside it. Transaction read concern is set at transaction start; commit uses transaction-level write concern. Long transactions retain resources/history and increase conflict/failure probability.

Transactions require replica-set or sharded-cluster support, not a standalone server.

Conflict/commit timeline

TEXT

```
attempt 1: read snapshot -> update -> transient conflict -> abort  
attempt 2: callback reruns -> update -> commit request -> network loss  
commit result: unknown -> retry commit/reconcile, not whole external workflow blindly
```

Failure matrix

Failure	State	Response
primary stepdown before write	likely no accepted write	driver retry if labeled/supported
timeout after send	unknown	reconcile by idempotency key
transient transaction error	attempt aborted	retry whole DB callback within budget
unknown transaction commit result	commit uncertain	retry commit according to driver contract
duplicate key	invariant conflict/prior request	interpret, do not generic retry
stale secondary read	allowed by routing/lag	stronger route/session if contract needs it

Practice and solution direction

- **Foundation:** Separate read preference from read concern.
- **Interview Core:** Select guarantees for order confirmation versus recommendation feed.
- **SDE-2 Follow-up:** Design a transaction callback that writes an order and event without emailing twice. Persist an outbox document in the transaction; deliver idempotently afterward.

SDE-2 CORE CHAPTER

Chapter 7 - Sharding, Change Streams, Migrations, Operations, and Security

Sharding partitions data by key ranges or hashed space

A sharded cluster routes operations through `mongos` to shards according to collection metadata. The shard key affects distribution, query targeting, migration cost, and the ability to enforce some uniqueness constraints.

Shard-key worksheet

Evaluate candidates by:

- cardinality and frequency distribution;
- monotonicity and write hot spots;
- query routing prefixes;
- ability to add zones/locality if needed;
- document growth and chunk movement;
- immutable/stable availability in every operation;
- uniqueness requirements;
- future resharding cost.

`customerId` can co-locate a customer's orders and target customer queries, but a celebrity tenant may create a hot shard. A hashed key spreads writes but loses natural range locality. A compound key may balance both; test the actual workload.

Targeted versus scatter-gather

If a query includes the shard key or a usable prefix/range, the router can target fewer shards. Without it, the query may scatter, merge, sort, and limit results across shards. A fast single-shard plan does not predict cluster-wide behavior.

Chunk migration and balancing

Balancing moves data to distribute load/storage. Migration consumes network, I/O, and coordination and can interact with long-running work. Observe shard imbalance, jumbo/large ranges, migration duration, and tail latency. Do not schedule major index builds, backfills, and rebalancing together without capacity evidence.

Change streams

Change streams expose committed changes from the oplog through resumable cursors. They are useful for projections, cache invalidation, and integrations, but consumers must handle:

- duplicate delivery after resume/retry;
- resume-token persistence after durable processing;
- retention gaps/invalidation requiring rebuild;
- ordering scope rather than assuming global order;
- schema evolution and large-event handling;
- idempotent destination writes.

Persisting the resume token before the side effect can lose work; after the side effect can repeat it. Use idempotent writes or an atomic destination checkpoint when possible.

Schema migration in a flexible store

Flexible documents make gradual rollout possible, not effortless:

- 1 deploy readers compatible with old/new shape;
- 2 write the new shape and stable version marker;
- 3 backfill in bounded restartable ranges;
- 4 verify counts/domain invariants and index usage;
- 5 tighten collection validation;
- 6 remove old compatibility only after observation.

Avoid rewriting every document merely to add a value that can be interpreted as a default on read-unless queries/indexes/validation require materialization.

Backup and restore

Replication is not backup. Define RPO/RTO, use consistent snapshots plus oplog/point-in-time mechanisms appropriate to the deployment, encrypt and isolate backups, and restore-test the application invariants. Sharded recovery needs cluster-consistent planning, not independent ad hoc shard copies.

Observability

Monitor:

- driver pool wait and server-selection time;
- command latency/errors by normalized shape;
- documents/keys examined versus returned;
- replication lag and election frequency;
- cache pressure, tickets/queues, disk and journal latency;
- locks/transactions and oldest active work;
- shard targeting, imbalance, migrations, chunk growth;
- change-stream lag/resume failures;
- backup age and restore-test results.

Security baseline

Authenticate services, use least-privilege database roles, restrict networks, enable TLS, rotate secrets, encrypt backups, audit privileged operations, redact query values, and validate user-controlled operators. Do not deserialize arbitrary client JSON directly into a MongoDB query; operator injection can bypass the intended filter.

Operational edge matrix

Incident	Evidence	First correction
one shard at 95% CPU	shard-key distribution and targeting	remove scatter/hot-key cause before adding nodes
change consumer missed history	resume token and oplog retention	rebuild projection then resume from safe point
backfill hurts writes	batch size/yield/index use	throttle and checkpoint
repeated elections	member/network/disk health	fix infrastructure and acknowledgement assumptions
backup "green," restore unusable	restore logs/domain checks	automated restore drills

Practice

- **Interview Core:** Choose a shard key for multi-tenant orders and name the hot-tenant failure.
- **SDE-2 Follow-up:** Build a cache-invalidation consumer that survives duplicate events and expired resume tokens.

SDE-2 CORE CHAPTER

Chapter 8 - Java Driver and Spring Data Boundaries, Testing, and Diagnostics

Driver command shape first

JAVA

```
Bson filter = and(
    eq("_id", orderId),
    eq("status", "CREATED"),
    eq("version", expectedVersion));

Bson update = combine(
    set("status", "PAID"),
    currentDate("paidAt"),
    inc("version", 1L));

UpdateResult result = orders.updateOne(filter, update);
if (result.getMatchedCount() != 1) {
    throw new StaleOrderException(orderId);
}
```

The filter is the concurrency contract. `modifiedCount` can be zero for an idempotent update that matched but changed no stored value; use the count that matches the business question.

Client, pool, and codec lifecycle

`MongoClient` owns pools and topology monitoring and is normally shared. A collection handle is lightweight and carries database, codec, read/write concern, and preference settings. Do not create a new client per request.

Codecs convert BSON and Java. Test optional/missing fields, numeric width, enum evolution, dates, unknown fields, and constructor/default behavior. A mapping exception during cursor iteration can occur after the query succeeded.

Timeouts and server selection

Different budgets cover server selection, pool checkout, connection establishment, socket I/O, and server execution (`maxTimeMS`). One giant timeout hides where latency occurs. Cancellation/timeout can leave an unknown write result; preserve an operation/request key.

Spring Data progression

- 1 Use native `MongoCollection` to learn the exact command.
- 2 Use `MongoTemplate` when filters, updates, aggregation, transactions, or result metadata must remain explicit.
- 3 Use repositories for honest CRUD/derived-query contracts.

JAVA

```
Query query = Query.query(Criteria.where("id").is(id)
    .and("status").is(CREATED)
    .and("version").is(expectedVersion));
Update update = new Update()
    .set("status", PAID)
    .currentDate("paidAt")
    .inc("version", 1);
UpdateResult result = mongoTemplate.updateFirst(query, update, OrderDocument.class);
```

Repository `save` may replace broadly and is not a conditional state-transition API. For concurrency-sensitive partial updates, keep filter, operators, counts, and concern visible.

Transactions in Spring

Spring transaction annotations require a configured Mongo transaction manager and a replica set/sharded cluster. Self-invocation/proxy rules still apply. A transaction does not include a remote API or Kafka unless a separate coordination design exists.

Testing layers

- pure Java tests for model/filter/cursor/retry decisions;
- BSON command-shape tests without a server;
- driver integration tests against an exact MongoDB replica set;
- Spring tests for mapping, transactions, and exception translation;
- production telemetry for plans, lag, pool waits, and failures.

An embedded or substitute database cannot prove target-version sharding, elections, transactions, or query plans.

Diagnostic sequence

TEXT

```
request span
-> pool/server-selection duration
-> command name + normalized filter/comment
-> chosen member/shards
-> execution stats and waits
-> cursor batches + decode time
```

Use command comments/trace correlation carefully without putting secrets in logs.

WATCH OUT | Java edge cases

Mistake	Failure
client per request	connection/topology churn
<code>toJson()</code> logs full document	PII/secrets leakage
compare <code>modifiedCount == 1</code> for idempotent set	false conflict
repository replacement with stale object	lost/new-field deletion risk
retry whole callback with HTTP side effect	duplicate external action
block reactive driver on event loop	throughput collapse

Practice and solution direction

- **Foundation:** Build a typed filter/update for one state transition.
- **Interview Core:** Decide `matchedCount` versus `modifiedCount` for idempotent cancellation.
- **SDE-2 Follow-up:** Add traceable, bounded retry without exposing document values; log command shape, labels, attempt, server-selection/pool time, and request key hash.

SDE-2 CORE CHAPTER

Chapter 9 - MongoDB Live Interviews, Rapid Q&A, Practice, and Sources

Live interview 1: embed or reference

Interviewer: "Model orders and order lines."

Candidate: "Lines are bounded, owned, read with the order, and participate in one invariant, so I would embed them. Product and customer remain references; agreed price/address are snapshots. I would cap line count and reject growth near the document limit."

Follow-up: "What if an order has millions of events?"

Worked answer: Keep current order state and a bounded recent window; store immutable events separately or in buckets indexed by order/time.

Live interview 2: duplicate checkout

Interviewer: "Two identical create requests arrive."

Candidate: "Use (`customerId`, `requestKey`) as a unique index and insert once. A duplicate-key result maps to the original request. A check-then-insert query is not race safe."

Follow-up: "The network timed out."

Worked answer: The outcome is unknown. Reconnect and query the idempotency key rather than generate a new key.

Live interview 3: slow array query

Interviewer: "Find orders containing SKU A with quantity at least 10, but results are wrong."

Candidate: "Independent dot predicates may match different array elements. Use `$elemMatch` and test a compound multikey index under representative array sizes. I would inspect keys/documents examined and avoid projecting the whole large array if not needed."

Live interview 4: newest orders pagination

Interviewer: "Page 500,000 is slow."

Candidate: "Replace deep `skip` with keyset order (`createdAt DESC, _id DESC`), store both values in the cursor, and align a compound index after equality filters. Define concurrent insert/delete behavior and cursor tamper protection."

Live interview 5: multi-document checkout

Interviewer: "Update order, inventory, and emit an event."

Candidate: "First see whether inventory can use atomic conditional updates and reservation state. If order/outbox must commit together, use a short MongoDB transaction on a replica set, with an idempotent callback. External publication happens from the outbox after commit. Cross-document inventory conflict requires retry or compensation."

Live interview 6: stale confirmation

Interviewer: "Write succeeds, immediate read says not found."

Candidate: "Trace member selection and session. A secondary may lag. Use primary read or causal session/appropriate concern for confirmation; keep stale secondary reads for flows with an explicit staleness budget."

Live interview 7: shard hot spot

Interviewer: "One shard handles almost every write."

Candidate: "Inspect shard-key distribution and monotonicity. A timestamp/range or celebrity tenant can concentrate traffic. Compare hashed/compound candidates against targeted query needs, uniqueness, migration, and hot-tenant isolation; do not reshard without load rehearsal and rollback criteria."

Rapid answered questions

- 1 **Is MongoDB schema-less?** No; documents have shape/types. Validation may live in app/database and evolves over time.
- 2 **Atomicity boundary?** One document for ordinary writes; multi-document transactions are available with deployment requirements/cost.
- 3 **Missing versus null?** Distinct stored states; common null queries can match both unless `$exists` is used.
- 4 **Why unique index?** It enforces a concurrent invariant; an application pre-check does not.
- 5 **Replacement versus `$set`?** Replacement removes omitted fields; `$set` changes selected paths.
- 6 **Why `$elemMatch`?** It requires several predicates to match the same array element.
- 7 **Is TTL exact?** No; background deletion can be delayed. Enforce security expiry when reading/authorizing.
- 8 **What is a multikey index?** An index over array values where one document contributes multiple keys.
- 9 **Does `IXSCAN` mean fast?** No; compare keys/docs examined, returned rows, sort, fetch, and latency.
- 10 **Read preference versus concern?** Member routing versus consistency/isolation contract.
- 11 **Does majority mean latest?** It means majority-committed data, not necessarily newest known data everywhere.
- 12 **Why can transaction callback repeat?** Drivers may retry transient transaction attempts; external effects must not live inside unprotected.
- 13 **Replica set versus backup?** Replication improves availability; it reproduces mistakes and lacks historical recovery alone.
- 14 **Why not unbounded arrays?** Document limit, growing read/write cost, and hot-document contention.
- 15 **Change streams exactly once?** Design for resume and duplicates; destination idempotency/checkpointing determines outcome.
- 16 **`matchedCount` versus `modifiedCount`?** Match proves predicate found a document; modification says stored bytes/value changed.

Cumulative assessment and solution rubric

Design a multi-tenant order store with idempotent creation, embedded lines, cursor search, inventory reservation, change-stream projection, and sharding. Deliver document schemas, validators, four indexes, command shapes, concern/routing decisions, retry timelines, shard-key analysis, restore plan, and dashboards.

Strong solution: bounds embedded growth, encodes state/version in updates, uses unique request keys, explains multikey/index plans, keeps transactions short and external work in an outbox, selects consistency per endpoint, makes projections rebuildable/idempotent, and identifies hot-tenant/scatter risks.

Authoritative references

- MongoDB Manual: data modeling, CRUD, indexes, aggregation, replication, [read concern](#), transactions, sharding, change streams, backup, and security.
- MongoDB Java Sync Driver documentation: clients, codecs, filters/updates, sessions, transactions, concerns, and monitoring.
- Spring Data MongoDB reference: mapping, [MongoTemplate](#), repositories, transactions, indexes, and observability.

The companion lab validates Java driver BSON command construction without a server. Target-version replica-set/sharded tests remain mandatory for actual plans, elections, transactions, and routing.

SDE-2 CORE CHAPTER

Chapter 10 - Java 21 MongoDB Interview Reasoning Companion

Executable models for document boundaries, optimistic filters, cursor ordering, transaction retry labels, and idempotent change projections.

JAVA (continued 1/3)

```
import java.time.Instant;
import java.util.Comparator;
import java.util.HashSet;
import java.util.List;
import java.util.Objects;
import java.util.Set;

/** Dependency-free executable reasoning models for the MongoDB volume. */
public final class MongoDBInterviewCompanion {
    private MongoDBInterviewCompanion() {}

    enum Boundary { EMBED, REFERENCE }

    record Relationship(boolean owned, boolean bounded, boolean readTogether,
        boolean independentlyQueried) {}

    static Boundary chooseBoundary(Relationship relationship) {
        Objects.requireNonNull(relationship, "relationship");
        return relationship.owned()
            && relationship.bounded()
            && relationship.readTogether()
            && !relationship.independentlyQueried()
            ? Boundary.EMBED : Boundary.REFERENCE;
    }

    record VersionedDocument(String status, long version) {
        VersionedDocument transition(String requiredStatus, long expectedVersion, String nextStatus) {
            return status.equals(requiredStatus) && version == expectedVersion
                ? new VersionedDocument(Objects.requireNonNull(nextStatus), version + 1)
                : this;
        }
    }

    record Cursor(Instant createdAt, String objectIdHex) {
```

JAVA (continued 2/3)

```
    Cursor {
        Objects.requireNonNull(createdAt, "createdAt");
        Objects.requireNonNull(objectIdHex, "objectIdHex");
    }
}

static final Comparator<Cursor> NEWEST_FIRST = Comparator
    .comparing(Cursor::createdAt, Comparator.reverseOrder())
    .thenComparing(Cursor::objectIdHex, Comparator.reverseOrder());

static boolean comesAfter(Cursor candidate, Cursor boundary) {
    return NEWEST_FIRST.compare(candidate, boundary) > 0;
}

enum RetryAction { DO_NOT_RETRY, RETRY_TRANSACTION, RETRY_COMMIT, RECONCILE }

static RetryAction classify(Set<String> errorLabels, boolean writeOutcomeKnown) {
    Set<String> labels = Set.copyOf(errorLabels);
    if (labels.contains("UnknownTransactionCommitResult")) {
        return RetryAction.RETRY_COMMIT;
    }
    if (labels.contains("TransientTransactionError")) {
        return RetryAction.RETRY_TRANSACTION;
    }
    return writeOutcomeKnown ? RetryAction.DO_NOT_RETRY : RetryAction.RECONCILE;
}

static final class IdempotentProjection {
    private final Set<String> appliedEventIds = new HashSet<>();
    private long totalCents;

    boolean apply(String eventId, long deltaCents) {
        Objects.requireNonNull(eventId, "eventId");
        if (!appliedEventIds.add(eventId)) {

```

JAVA (continued 3/3)

```

        return false;
    }
    totalCents = Math.addExact(totalCents, deltaCents);
    return true;
}

public static void main(String[] args) {
    assert chooseBoundary(new Relationship(true, true, true, false)) == Boundary.EMBED;
    assert chooseBoundary(new Relationship(true, false, true, false)) == Boundary.REFERENCE;

    VersionedDocument document = new VersionedDocument("CREATED", 3);
    assert document.transition("CREATED", 2, "PAID").equals(document);
    assert document.transition("CREATED", 3, "PAID").version() == 4;

    Instant tied = Instant.parse("2026-08-02T10:00:00Z");
    Cursor boundary = new Cursor(tied, "00000000000000000000ff");
    assert comesAfter(new Cursor(tied, "00000000000000000000fe"), boundary);

    assert classify(Set.of("TransientTransactionError"), true)
        == RetryAction.RETRY_TRANSACTION;
    assert classify(Set.of("UnknownTransactionCommitResult"), false)
        == RetryAction.RETRY_COMMIT;
    assert classify(Set.of(), false) == RetryAction.RECONCILE;

    IdempotentProjection projection = new IdempotentProjection();
    assert projection.apply("event-1", 500);
    assert !projection.apply("event-1", 500);
    assert projection.totalCents == 500;

    System.out.println("MongoDbInterviewCompanion checks passed");
}
}

```


Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: documents and CRUD
- Interview Core: models, indexes, and aggregation
- SDE-2 Follow-up: consistency, scaling, and integration

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

[Previous book: FW 06 - Spring Data for Java Backend Engineers](#)

[Series index](#)

[Next book: FW 08 - Redis for Java Backend Interviews](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	JAVA 01	Java Foundations for Problem Solving
Java	JAVA 02	Git and GitHub for Java Engineers
Java	JAVA 03	Maven and Gradle for Java Engineers
Java	JAVA 04	Language, OOP, and Modern Java
Java	JAVA 05	Collections, Streams, and I/O
Java	JAVA 06	JVM and Java Execution
Java	JAVA 07	Java Concurrency and the Memory Model
Java	JAVA 08	Performance, Diagnostics, and GC Incidents
Java	JAVA 09	Java Question Bank, Study Plan, and Reference
DSA	DSA 01	Time and Space Complexity for Java Interviews
DSA	DSA 02	Number Systems and Math Foundations for DSA Interviews
DSA	DSA 03	Number Systems Interview Patterns and Rapid Revision
DSA	DSA 04	Bit Manipulation in Java
DSA	DSA 05	Loop Mastery, Patterns, and Index Calculations
DSA	DSA 06	Arrays and Array Problem-Solving Patterns
DSA	DSA 07	Strings and String Problem-Solving Patterns
DSA	DSA 08	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	DSA 09	Recursion and Backtracking in Java
DSA	DSA 10	Linked Lists: Pointer Reasoning and Mutation
DSA	DSA 11	Stacks, Queues, Deques, and Monotonic Patterns
DSA	DSA 12	Binary Search: Bounds, Answers, and Invariants
DSA	DSA 13	Trees, Binary Search Trees, and Tries
DSA	DSA 14	Heaps, Priority Queues, Selection, and Top-K
DSA	DSA 15	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	DSA 16	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	DSA 17	Dynamic Programming: State, Transitions, and Optimization
Frameworks	FW 01	MySQL for Java Backend Interviews
Frameworks	FW 02	Hibernate and JPA for Java Backend Interviews
Frameworks	FW 03	Spring Framework for Java Engineers
Frameworks	FW 04	Spring Boot for Java Backend Engineers
Frameworks	FW 05	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	FW 06	Spring Data for Java Backend Engineers
Frameworks	FW 07	MongoDB for Java Backend Interviews
Frameworks	FW 08	Redis for Java Backend Interviews
Frameworks	FW 09	Persistence, SQL, JPA, and Caching
Frameworks	FW 10	Apache Kafka and Spring Kafka
Frameworks	FW 11	Spring Ecosystem Extensions
Frameworks	FW 12	Spring AI for Java Engineers
System Design	SD 01	Design, Backend, Testing, and Security
System Design	SD 02	Distributed Systems and System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Frameworks, Data, and Messaging - Book 07 of 12 - Study Step FW-07. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.